

CS 4032 – Web Programming



JavaScript

JavaScript to program the behavior of web pages



JavaScript - Introduction

- JavaScript is client-side programming language
 - developed by Brendan Eich at Netscape
- Communicates with web browser
- Allows user-web page interactivity
- JavaScript cannot be forced on users web browsers
 - Must be supported by web browsers
 - Most web browsers support JavaScript by default
- JavaScript is not JAVA
- JavaScript is an object-oriented language



What can JavaScript do?

- JavaScript can dynamically modify an HTML page
- JavaScript can react to user input
- JavaScript can validate user input
- JavaScript can be used to create cookies
- JavaScript is a full-featured programming language
- JavaScript user interaction does not require any communication with the server

How to include JavaScript in html

- JavaScript programs require the <SCRIPT> tag in .html files

<script type = "text/javascript">

ACTUAL JavaScript code here

</script>

- These can appear in either the <HEAD> or <BODY> section of an html document
 - Functions and code that may execute multiple times is typically placed in the <HEAD>
 - These are only interpreted when the relevant function or event-handler are called



Simple User Interaction

- There are three built-in methods of doing simple user interaction
 - `alert(msg)` alerts the user that something has happened
 - `confirm(msg)` asks the user to confirm (or cancel) something
 - `prompt(msg, default)` asks the user to enter some text
 - `console.log(msg)` — Writes information to the browser console, good for debugging purposes
 - `document.write(msg)` — Write directly to the HTML document

```
alert("There's a monster on the wing!");  
  
confirm("Are you sure you want to do that?");  
  
prompt("Enter you name", "Adam");
```

Simple Example

```
<HTML>
<HEAD>
<TITLE>First JavaScript Example</TITLE>
</HEAD>
<BODY>
<H2>This line is straight HTML</H2>
<H3>
<SCRIPT type = "text/javascript">
    document.write("These lines are produced by<br/>");
    document.write("the JavaScript program<br/>");
    alert("Hey, JavaScript is fun!");
</SCRIPT>
</H3>
<H2>More straight HTML</H2>
</BODY>
</HTML>
```

JavaScript - Variables

- JavaScript variables have no types
 - Type is **determined dynamically**, based on the value stored
 - This is becoming familiar!
 - The **typeof** operator can be used to check type of a variable
- Declarations are made using the **var** or **let** keyword
 - Can be implicitly declared, but not advisable
 - Declarations **outside** of any **function** are **global**
 - Declarations **within** a **function** are **local** to that function
 - Variables declared but not initialized have the value **undefined**
- Variable identifiers are like those in other languages (ex: Java)
 - Cannot use a keyword
 - Must begin with a letter, \$, or _
 - Followed by any sequence of letters, \$, _ or digits
 - Case sensitive

JavaScript Expressions

- Numeric operators in JavaScript are similar to those in most languages
 - + , − , * , / , % , ++ , --
 - Precedence and associativity are also fairly standard
- Strings
 - Have the + operator for concatenation
 - Have a number of methods to do typical string operations
 - **charAt, indexOf, toLowerCase, substring**
 - Looks kind of like Java – intentionally



JavaScript Control Statements

- JavaScript has most of the operators that we used in C++/Java
 - Arithmetic (+, -, *, /, %)
 - Assignment (=, +=, -=, *=, /=, %=, ++, --)
 - Logical (&&, ||, !)
 - Comparison (<, >, <=, >=, ==)
- Constructs:
 - if, else, switch, case
 - while, do-while, for, for-in, for-each



JavaScript Functions

- JavaScript lets you define functions using the function keyword
- Functions can return values using the return keyword

```
function showConfirm() {  
    confirm("Are you sure you want to do that?");  
}
```

JavaScript Arrays

- JavaScript has arrays that are indexed starting at 0
- Special version of for works with arrays

```
<script type="text/javascript">
  var colors = new Array();
  colors[0] = "red"; colors[1] = "green";
  colors[2] = "blue"; colors[3] = "orange";
  colors[4] = "magenta"; colors[5] = "cyan";
  for (var i in colors) {
    document.write("<div style=\"background-color:\"
                  + colors[i] + \";\>\"
                  + colors[i] + "</div>\n");
  }
</script>
```

JavaScript Events

- JavaScript can be made to respond to user events
- Common Events:
 - onload and onunload : when a page is first visited or left
 - onfocus, onblur, onchange : events pertaining to form elements
 - onsubmit : when a form is submitted
 - onmouseover, onmouseout : for "menu effects"



Exception Handling

- JavaScript also has try, catch, and throw keywords for handling JavaScript errors.

```
try {
    runSomeCode();
} catch(err) {
    var txt="There was an error on this page.\n\n"
        + "Error description: "
        + err.description + "\n\n"
    alert(txt)
}
```

Comments in JavaScript

- Comments in JavaScript are delimited with `//` and `/* */` as in Java and C++



JavaScript Objects

- Some basic objects are built-in to JavaScript
 - **String – Date – Array – Boolean – Math**



JavaScript Objects

- Objects are certain kinds of variables
- They are variables that can have their own values and methods

```
var person = {  
  firstName:"John",  
  lastName:"Doe",  
  age:20,  
  nationality:"German"  
};
```

JavaScript Strings

- A String object is created every time you use a string literal (just like in Java)
- Have many of the same methods as in Java
 - `charAt`, `concat`, `indexOf`, `lastIndexOf`, `match`, `replace`, `search`, `slice`, `split`, `substr`, `substring`, `toLowerCase`, `toUpperCase`, `valueOf`
- There are also some HTML specific methods
 - `big`, `blink`, `bold`, `fixed`, `fontcolor`, `fontsize`, `italics`, `link`, `small`, `strike`, `sub`, `sup`
- Don't use the HTML methods (use CSS instead)
 - This is the worst kind of visual formatting

JavaScript Dates

- The Date class makes working with dates easier
- A new date is initialized with the current date
- Dates can be compared and incremented

```
var myDate = new Date();  
myDate.setFullYear(2007,2,14);  
  
var today = new Date();  
var nextWeek = today + 7;  
  
if (nextWeek > today) {  
    alert("You have less than one week left");  
}
```

JavaScript and the DOM

- Using DOM, JavaScript can perform multiple tasks
 - create new elements and attributes
 - change the existing elements and attributes
 - remove existing elements and attributes
- JavaScript can also react to existing events and create new events in the page

HTML DOM: Document

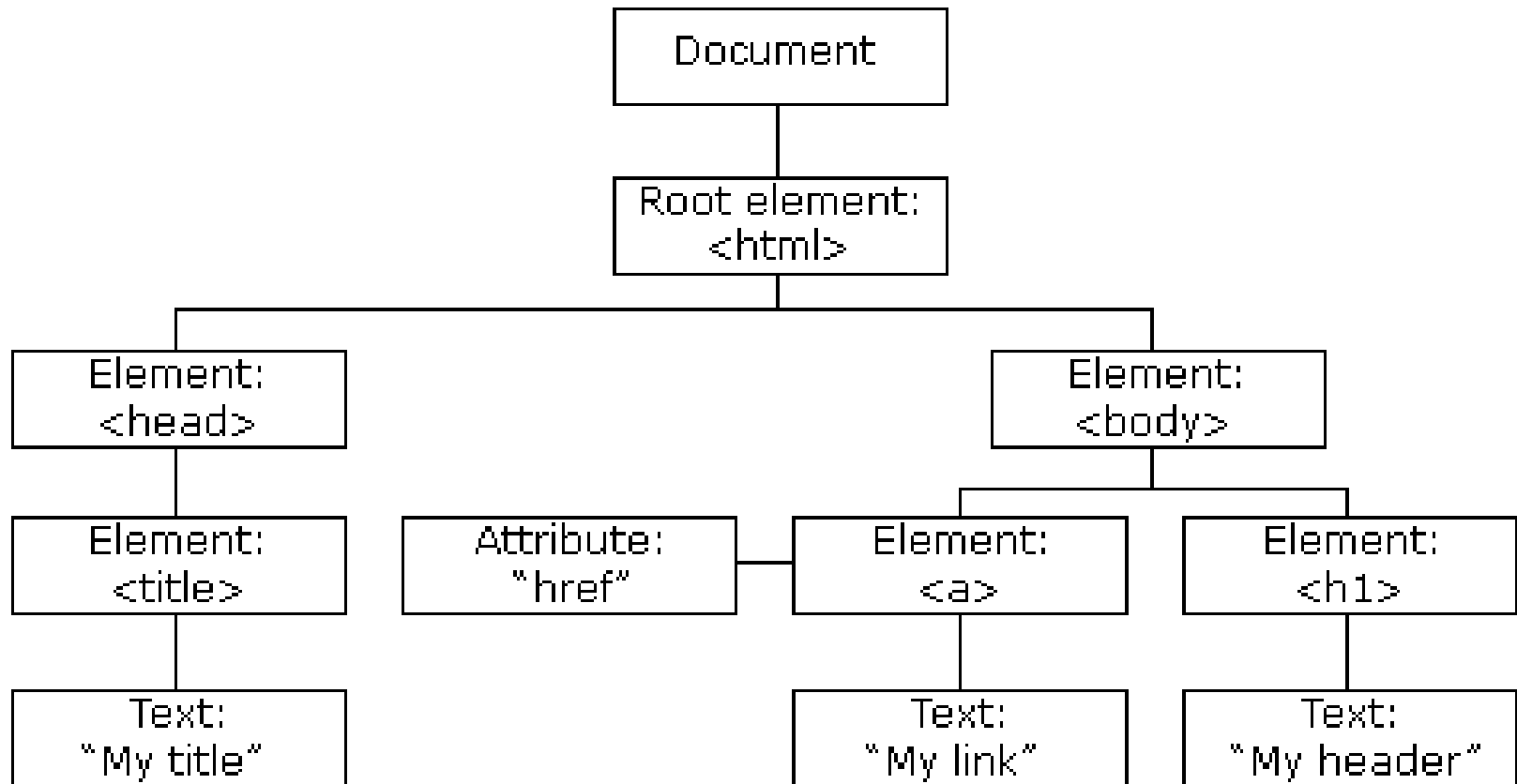
- The Document object represents an HTML document and can be used to access all documents in a page
- A Document contains several collections
 - anchors, forms, images, links
- Has several properties
 - name, id, title, URL, ref, etc.



HTML DOM: Objects

- Environment objects
 - Window, Navigator, Screen, History, Location, Document
- HTML objects
 - Anchor, Area, Base, Body, Button, Event, Form, Frame, Frameset, Iframe, Image, Checkbox, FileUpload, Hidden, Password, Radio, Reset, Submit, Text, Link, Meta, Object, Option, Select, Style, Table, TableCell, TableRow, TextArea

HTML DOM: The Document Tree



Dynamic HTML (DHTML)

- Manipulating the web page's structure is essential for creating a highly responsive UI
 - Dynamic HTML content modification
- Two main approaches
 - Manipulate page via plain JS
 - Manipulate page using JS + library (e.g., jQuery)

Navigating the Document Tree

- With JavaScript we can navigate the document tree
 - `document.getElementById()`
 - `document.getElementsByName()`
 - `document.getElementsByTagName()` return nodes in the document tree
 - `document.innerHTML`: To access the content of an element
 - `document.createElement`: To create new element
 - `document.createAttribute`: To create new attribute
 - `document.removeChild`: Remove an element
 - `Document.appendChild`: Appends an element
- Information can be obtained from
 - `nodeName` – The tag name
 - `nodeValue` – The text of a text node
 - `nodeType` – The kind of node

Rewriting using innerHTML attribute

```
<span id="stuff"></span>
```

```
<form><input id="inpt" onchange="doit()"></form>
```

```
<script>
```

```
function doit() {
```

```
    document.getElementById("stuff").innerHTML =
```

```
    document.getElementById("inpt").value;
```

```
}
```

```
</script>
```



10
10

Rewriting the contents of a span. NOTE: There is a security problem in the code above.

Assigning the .innerText instead

```
<span id="stuff"></span>
<form><input id="inpt" onchange="doit()"></form>
<script>
function doit() {
    document.getElementById("stuff").innerText =
    document.getElementById("inpt").value;
}
</script>
```



Rewriting the contents of a span. NOTE: There is a browser-compatibility problem in the code above.

Example

```
<html>
<head>
  <title>DOM!!!</title>
</head>
<body>
  <h1 id="one">Welcome</h1>
  <p>This is the welcome message.</p>
  <h2>Technology</h2>
  <p>This is the technology section.</p>
  <script type="text/javascript">
    var text = document.getElementById("one").innerHTML;
    alert("The first heading is " + text);
  </script>
</body>
</html>
```

Event handler

- You can add an **event handler** to a particular element like this

```
document.getElementById(id).onclick=function()  
{  
    lines of code to be executed  
}
```

OR

```
document.getElementById(id).addEventListener("click", functionname)
```

```
<html>  
<head>  
    <title>DOM!!!</title>  
</head>  
<body>  
    <input type="button" id="btnClick" value="Click Me!!" />  
    <script type="text/javascript">  
        document.getElementById("btnClick").addEventListener("click", clicked);  
        function clicked()  
        {  
            alert("You clicked me!!!");  
        }  
    </script>  
</body>  
</html>
```



JSON: JavaScript Object Notation

- Defines a format to store information in an organized and easy-to-access way
- Douglas Crockford specified the JSON format in the early 2000s.
- The official website was launched in 2002.
- JSON became an ECMA international standard in 2013.
- The most updated JSON format standard was published in 2017.



JSON Data Types

- String
 - E.g., { "name": "John" }
- Number
 - E.g., { "age": 25 }
- Object (JSON object)
 - E.g., { "employee": { "name": "John", "age": 30, "city": "New York" } }
- Array
 - E.g., { "employees": ["John", "Anna", "Peter"] }
- Boolean
 - E.g., { "active": true }
- Null
 - E.g., { "middlename": null }

Parsing JSON

- The data received from server is always in the form of a string
- Parse the data with `JSON.parse()`
 - Returns a JavaScript object
- E.g.,
 - `var obj = JSON.parse('{ "name":"John", "age":30, "city":"New York"}');`
 - `obj.name, obj.age, obj.city`
- If the server response is a JavaScript array
 - `JSON.parse()` will return an array

How to convert JavaScript object to JSON text?

- By using method `JSON.stringify()`
 - Converts JavaScript object to string
 - `var obj = { name: "John", age: 30, city: "New York" };`
 - `var jsonText= JSON.stringify(obj);`
- It is also possible to stringify JavaScript arrays

Simple Example in JSON

- The following example shows how to use JSON to store information related to books based on their topic and edition.
- It basically has key-value pairs.

```
var Book= {  
    "id" : "01",  
    "language" : "Java",  
    "edition" : "third",  
    "author" : "Herbert Schildt"  
};  
document.writeln("The Language is: " + Book.language);  
document.writeln("its Edition is: " + Book.edition);  
document.writeln("its Author is: " + Book.author);  
var myJSObject = JSON.parse(Book);
```

```
{  
  "book": [  
    {  
      "id": "01",  
      "language": "Java",  
      "edition": "third",  
      "author": "Herbert Schildt"  
    },  
    {  
      "id": "07",  
      "language": "C++",  
      "edition": "second",  
      "author": "E. Balagurusamy"  
    }  
  ]  
}
```

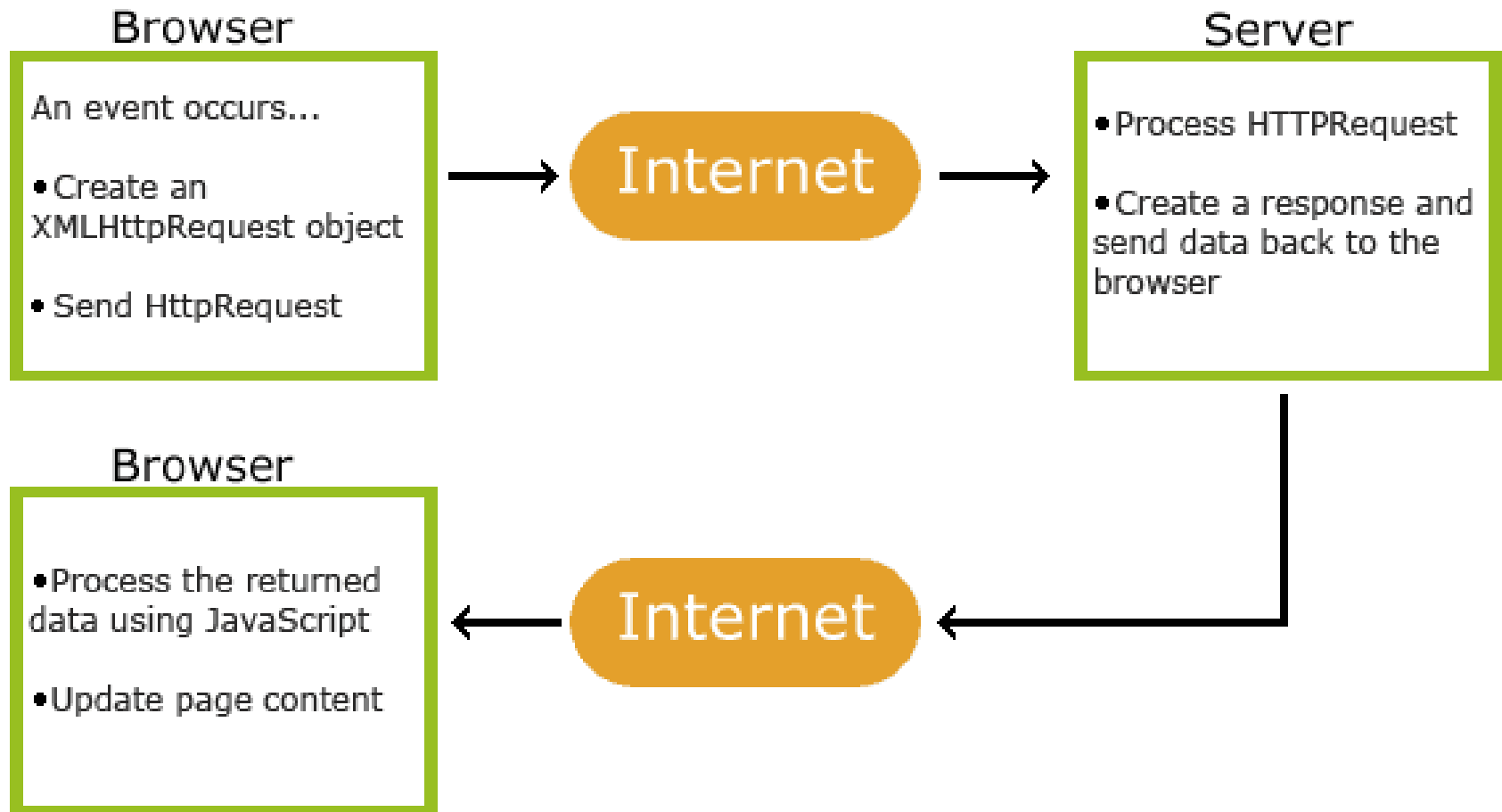
AJAX

- AJAX - Asynchronous JavaScript and XML
- AJAX is not a programming language
- AJAX is a way of using existing standards (JavaScript and XML) to make more interactive web applications
- AJAX was popularized in 2005 by Google (with Google suggest)
- Can be used to update page content without refreshing the page
 - Pages update asynchronously
 - Google Maps, Gmail

AJAX Technologies

- AJAX is based on web development technologies
 - **JavaScript:** The programming language used to develop Ajax applications
 - **XML:** Provides a means of exchanging structured data between the web server and client
 - **The XMLHttpRequest object:** Provides the ability to asynchronously exchange data between web browsers and a web server
 - **HTML and CSS:** Provides the ability to mark up and style the display of web page text
 - **The DOM:** Provides the ability to dynamically interact with and alter the web page layout and content

How AJAX Works



AJAX - The XMLHttpRequest Object

- The XMLHttpRequest object can be used to exchange data with a server
- Syntax for creating an XMLHttpRequest object:
 - `variable = new XMLHttpRequest();`
- Old Versions of Internet Explorer (IE5 and IE6)
 - Use an ActiveX object instead of the XMLHttpRequest object:
 - `variable = new ActiveXObject("Microsoft.XMLHTTP");`

AJAX - Send a Request

Method	Description
<code>open(<i>method</i>, <i>url</i>, <i>async</i>)</code>	Specifies the type of request <i>method</i> : the type of request: GET or POST <i>url</i> : the server (file) location <i>async</i> : true (asynchronous) or false (synchronous)
<code>send()</code>	Sends the request to the server (used for GET)
<code>send(<i>string</i>)</code>	Sends the request to the server (used for POST)

AJAX - Server Response

Property	Description
onreadystatechange	Defines a function to be called when the readyState property changes
readyState	Holds the status of the XMLHttpRequest. 0: request not initialized 1: server connection established 2: request received 3: processing request 4: request finished and response is ready
status	200: "OK" 403: "Forbidden" 404: "Page not found"
statusText	Returns the status-text (e.g. "OK" or "Not Found")
responseText	Get the response data as a string
responseXML	Get the response data as XML data

A very simple web page and XML

```
<!DOCTYPE html><html><head>
<script src="jquery-1.8.2.min.js"></script>
</head><body>
<div id="clickme" onclick="startAjax()">Click
me</div>
<script>
function startAjax() {
    $("#clickme").text("Calling server");
    $.ajax({url:"somefile.xml",
        success:callbackFunction, error:errorFunction}
    );
}
function callbackFunction(data,info) {
    $("#clickme").text("result:"+data);
}
function errorFunction(data,info) {
    $("#clickme").text("error occurred:"+info);
}
</script>
</body></html>
```

result:

somefile.xml

```
<?xml version="1.0"?>
<root>
    <entry name="blah">ok</entry>
</root>
```

error occurred:error

Thank you!



National University
Of Computer and Emerging Sciences